

CAPSTONE PROJECT PROPOSAL

SOFTWARE ENGINEERING AND PROJECT MANAGEMENT LAB

Find Your Doctor

Tangail Doctor Directory and Appointment Booking Prototype

Submitted by	Md. Khalid Hasan
Student ID	IT24013
Course title	Software Engineering and Project Management Lab
Course code	ICT 3108
Submitted to	Professor Dr. Ziaur Rahman
Department	Information and Communication Technology
University	Mawlana Bhashani Science and Technology University
Proposal version	1.0 - Initial capstone proposal
Planning window	2 November 2026 - 15 January 2027 (proposed)

Abstract

Finding an appropriate doctor can require patients to search fragmented clinic information, compare specialities, understand visit schedules, and contact multiple facilities. *Find Your Doctor* is a lightweight web prototype that brings a doctor directory and a sample appointment workflow into one interface. The supplied baseline is implemented with HTML, CSS, and vanilla JavaScript. It includes doctor filters and profile cards, appointment booking, serial generation, patient status lookup, a demonstration admin panel, print/download slips, export and backup/restore actions. The current prototype stores records in the browser's `localStorage`; it has no server, shared database, authenticated service API, or real hospital integration. Its bundled listings and schedules must be independently validated before any real-world use.

This proposal defines the project objectives, boundaries, stakeholder needs, functional and non-functional requirements, architecture, six UML/analysis diagrams, a class-based interface prototype, risks, quality assurance, and an 11-week incremental delivery plan. A small object-oriented prototype is included to demonstrate seven classes that map directly to the class diagram. The proposed production direction is a server-backed system with controlled access and a normalized database, but the initial capstone deliverable prioritizes a testable, demonstrable student prototype and its documented design.

Contents

Abstract i

1 Project Overview 1

1.1 Project identity . . . 1
1.2 Executive summary . . . 1
1.3 Problem statement . . . 1
1.4 Proposed solution . . . 1

2 Goals, Objectives, and Success Criteria 1

2.1 Overall goal . . . 1
2.2 Specific objectives . . . 2
2.3 Acceptance criteria . . . 2

3 Scope and Boundaries 2

3.1 In scope for the capstone prototype . . . 2
3.2 Out of scope for this submission . . . 3
3.3 Assumptions and constraints . . . 3

4 Stakeholders and Use Context 3

4.1 Primary user journeys . . . 4

5 Requirements 4

5.1 Functional requirements . . . 4
5.2 Non-functional requirements . . . 5
5.3 Key business rules . . . 5

6 Software Process Model **5**
6.1 Selected model: Incremental development with short feedback cycles 5
6.2 Iteration strategy 6
6.3 Configuration and change control 6

7 Architecture and Technology **6**
7.1 Current prototype architecture 6
7.2 Technology selection rationale 7
7.3 Production target architecture (future extension) 7

8 Data and Domain Model **7**
8.1 Main data entities 7
8.2 Data quality and privacy 7

9 Analysis and Design Diagrams **8**

Data Flow Diagram - Context / Level 0 **9**

Data Flow Diagram - Level 1 **10**

Use Case Diagram **11**

Activity Diagram - Appointment Booking **12**

Entity-Relationship Diagram **13**

Class Diagram **14**

Object Diagram **15**

10 OOP Interface Prototype **16**
10.1 Implementation classes 16
10.2 How to run the OOP interface 16
10.3 Traceability 16

11 Project Schedule and Gantt Chart **16**
11.1 Planning calendar 16
11.2 Milestones and deliverables 18
11.3 Work breakdown structure 18
11.4 Deliverables register 18

12 Testing and Quality Assurance **19**
12.1 Test levels 19
12.2 Representative test cases 19
12.3 Defect severity 20

13 Risk Management **20**

14 Feasibility and Resource Plan **20**

14.1 Technical feasibility 20

14.2 Economic feasibility 21

14.3 Operational feasibility 21

14.4 Resources 21

15 Limitations and Future Enhancements **21**

15.1 Known limitations of the submitted prototype 21

15.2 Future roadmap 21

16 Ethics, Privacy, and Responsible Use **22**

17 Repository Structure and Reproducibility **22**

17.1 Reproducible run steps 22

18 Conclusion **22**

19 References **23**

A Submission Checklist **23**

1 Project Overview

1.1 Project identity

Project name	Find Your Doctor
Project type	Web-based doctor directory and appointment workflow prototype
Primary users	Patients/visitors and a demonstration administrator
Primary technologies	HTML5, CSS3, vanilla JavaScript
Current persistence	Browser `localStorage` and `sessionStorage`
Optional libraries	SheetJS for XLSX export; jsPDF for PDF appointment slips
Source organization	Plain HTML/CSS/JS, with no framework or server requirement
Delivery target	Responsive interface, working demo flow, documentation, UML/analysis diagrams, and test evidence

1.2 Executive summary

The project consolidates a doctor directory and a demonstrative appointment workflow in a single website. A visitor can filter doctor cards by hospital, department, speciality, and name; inspect a profile; choose a date; enter patient details; obtain a local appointment reference and serial; and check the locally stored appointment status. The demonstration admin panel supports appointment management and export/backup operations. An additional object-oriented page makes the core domain objects explicit for the course requirement.

The project uses an incremental process. Each iteration delivers a demonstrable slice and receives a review before the next. The submission repository includes the supplied website code organized into standard folders, editable Graphviz sources and rendered images for the diagrams, the LaTeX source and PDF proposal, a testing checklist, and run instructions.

1.3 Problem statement

Doctor information may be spread across clinic directories, social posts, phone calls, and printed schedules. A visitor can spend unnecessary time looking for specialities and visit details. A small digital directory can improve discoverability and demonstrate an appointment workflow, provided it clearly communicates data freshness and does not imply that a local browser record is a real hospital booking.

1.4 Proposed solution

Build and document a responsive web interface with searchable doctor cards, an appointment request form, serial/reference generation, appointment status viewing, and demo administrative functions. In the submitted scope these functions run in one browser and are intended for demonstration only. A production system would require a backend, verified master data, secure identity and access control, a shared database, audit logs, backups, and operational approval from participating facilities.

2 Goals, Objectives, and Success Criteria

2.1 Overall goal

Design, implement, and evaluate a usable web prototype that demonstrates how a doctor directory can connect to a basic appointment-request journey.

2.2 Specific objectives

1. Provide doctor search/filtering across name, hospital, department, and speciality.
2. Present key doctor information and a clear action to begin a booking.
3. Validate patient input and appointment dates before saving a demo appointment.
4. Generate a distinct local reference and sequential serial for the selected doctor, facility, and date.
5. Allow users to inspect the status of stored demo appointments and produce a printable/downloadable slip in the baseline application.
6. Provide demonstration administrator actions for confirming/cancelling, exporting, and backing up/restoring records.
7. Document structure and behaviour through DFD, use-case, activity, ERD, class, and object diagrams.
8. Demonstrate at least five classes through an interface that implements the class diagram.
9. Deliver source code, setup instructions, proposal PDF and editable LaTeX source in a public-ready repository.

2.3 Acceptance criteria

ID	Criterion	Evidence
AC-01	Search terms filter doctor records and empty results are explained.	Manual search tests
AC-02	A valid demo booking creates a unique reference and serial.	Booking test + browser record
AC-03	Invalid fields and past dates are rejected with clear messages.	Negative test cases
AC-04	Saved demo records remain visible after page refresh in the same browser origin.	Persistence check
AC-05	At least five implementation classes match the class diagram.	OOP prototype source + UML
AC-06	Every required analysis/design diagram is included and legible.	PDF/PNG/SVG + editable DOT sources
AC-07	PDF and LaTeX source are available at repository root.	Repository file check
AC-08	Limitations of local-only storage and sample data are prominently disclosed.	README, interface note, proposal

3 Scope and Boundaries

3.1 In scope for the capstone prototype

- Browse and filter the bundled doctor/facility dataset.
- View a doctor profile and schedule information supplied in the dataset.
- Demonstrate booking form validation, date selection, serial/reference generation, and an appointment slip.
- Demonstrate appointment lookup and status display using the records held by the browser.
- Demonstrate admin-facing appointment review, confirmation/cancellation, export and backup/restore in the baseline interface.

- Use local storage for persistence within one browser profile.
- Deliver an additional OOP prototype with at least seven classes and a mapping to the UML class diagram.
- Produce technical documentation, quality checks, diagrams, and a proposed completion schedule.

3.2 Out of scope for this submission

- Real-time integration with hospitals, doctors, national identity services, payment gateways, SMS, or email.
- Medical triage, diagnosis, treatment recommendations, emergency dispatch, or medical record management.
- A shared production database or cross-device synchronization.
- A claim that the bundled doctor names, affiliation, specialities, schedules, or phone/contact details are verified/current.
- Production-grade administrator authentication, role-based permissions, security monitoring, audit logging, or clinical-grade privacy compliance.

3.3 Assumptions and constraints

The existing source code supplied for this work is the baseline. The source guide describes a bundled dataset of 235 doctor/facility entries across 11 facilities and location data for Bangladesh; the data are treated as demonstration data until independently validated. Development assumes a modern browser and a local filesystem or VS Code Live Server. Optional XLSX/PDF libraries load from a CDN and may require internet access. The proposed schedule starts on 2 November 2026, subject to supervisor approval and availability of a reviewer for user feedback.

4 Stakeholders and Use Context

Stakeholder	Need / interest	Design response
Patient / visitor	Find relevant speciality and understand visit details quickly.	Search, filter, concise doctor cards, clear schedule notes, guided booking.
Demonstration admin	Review records and demonstrate status management.	Admin panel, pending/confirmed states, cancellation, export and backup.
Student developer	Meet course outcomes and show a complete software process.	Modular source, seven-class prototype, requirements, diagrams, tests, schedule.
Course instructor / reviewer	Assess proposal completeness and implementation traceability.	Acceptance criteria, milestone evidence, a reproducible repository and PDF.
Hospital/clinic representative (future)	Ensure correct listing and operational feasibility.	Required data verification step before any public deployment.

4.1 Primary user journeys

Journey A - Find a doctor: the visitor enters a name, facility, department, or speciality; the list updates; the visitor reviews profile/schedule information.

Journey B - Request an appointment: the visitor selects a doctor and date, enters patient details, submits the form, and receives a local reference/serial and a pending status. The interface makes clear that this is a demo record.

Journey C - Review a booking: a visitor checks a locally saved booking in the baseline site, or reviews demo appointment cards in the OOP page. Administrator features in the baseline demonstrate confirmation, cancellation, and export/backup workflows.

5 Requirements

5.1 Functional requirements

ID	Requirement	Description / acceptance
FR-01	Search/filter doctors	The user can filter available directory entries by hospital, department, speciality and doctor name. Filters combine to narrow results.
FR-02	Doctor detail	The user can open a profile/action view and see department, speciality, facility and visit schedule supplied by the dataset.
FR-03	Appointment request	The user can select a doctor/date and submit patient details after required-field validation.
FR-04	Serial and reference	The prototype assigns a local appointment reference and serial based on existing records for that doctor/facility/date.
FR-05	Appointment status	The baseline site provides a status lookup using registration number and mobile, with the final/current serial display.
FR-06	Appointment slip	The baseline site supports print and a PDF download when the optional PDF library is available.
FR-07	Admin management	The baseline demo supports login flow, confirmation/cancellation, and visible appointment records.
FR-08	Export and backup	The admin-facing tools can export appointment rows and save/restore a browser backup; XLSX depends on the optional CDN library.
FR-09	Browser persistence	Demo records are saved and retrieved from local browser storage.
FR-10	OOP interface	The added interface instantiates domain/service/UI classes and supports doctor search, booking, local persistence and list rendering.
FR-11	Responsiveness	The interface remains usable on desktop and narrow mobile screens.
FR-12	Safety disclosure	UI and documentation state that data are demonstrative and not a real appointment service.

5.2 Non-functional requirements

ID	Quality attribute	Requirement
NFR-01	Usability	Core tasks should be understandable without installation beyond opening the website; forms and errors use readable labels.
NFR-02	Performance	Filtering of the bundled dataset should feel responsive on a typical student laptop; no server round trip is required.
NFR-03	Compatibility	Target modern Chrome, Edge, and Firefox; optional CDN features may be unavailable offline.
NFR-04	Maintainability	HTML, CSS, data, and JavaScript responsibilities are separated; files and methods use descriptive names.
NFR-05	Accessibility	Use semantic headings, form labels, visible focus, live status messages, and keyboard-operable native controls.
NFR-06	Privacy	Only synthetic data should be used during evaluation. Browser storage is not a secure vault.
NFR-07	Reliability	Invalid JSON/storage failures should fail clearly where feasible; restore/export operations should be tested on copies.
NFR-08	Traceability	Requirements map to prototype features, diagrams, tests and milestones.

5.3 Key business rules

1. Doctor availability is derived from the schedule codes currently supplied with each record; “not specified” is not proof that a doctor is actually available.
2. The prototype caps appointments at 15 per doctor/facility/date in the demo logic. This capacity is configurable and must not be interpreted as a hospital policy.
3. An appointment starts in a pending state. The OOP page never claims to contact a facility or confirm a real appointment.
4. The original baseline uses registration/mobile matching for its local status flow. This is demonstration logic, not an appropriate production authentication mechanism.
5. All date/serial decisions in the current browser-only model are local and may be inconsistent across browsers or devices.

6 Software Process Model

6.1 Selected model: Incremental development with short feedback cycles

An incremental model is chosen because the project has separable features that can be designed, demonstrated and tested progressively. Each increment adds a coherent capability while keeping the previous version runnable. A short review at the end of each iteration gathers feedback from a supervisor or peer reviewer. The model is appropriate for an academic prototype with limited infrastructure; it does not eliminate the need for requirements, design, testing, and controlled releases.

6.2 Iteration strategy

Increment	Primary focus	Exit evidence
I1	Requirements and design	Scope, requirements catalogue, use cases, process model, initial diagram review.
I2	Search and profile UI	Responsive directory, filters, empty state, checked dataset format.
I3	Appointment workflow	Patient form, date rules, serial/reference generation, slip/status path.
I4	Admin and OOP interface	Demo management/export path, seven-class interface, diagrams aligned to code.
I5	Verification and release	Test results, defects resolved or recorded, final documentation, tagged demo package.

6.3 Configuration and change control

Git is the version-control system. Each meaningful change should have a descriptive commit. Requirements or schedule changes are logged in an issue or ‘CHANGELOG.md‘; before a milestone, the interface and documentation are checked together. A release candidate is not considered complete until the acceptance checklist, diagrams, and setup instructions agree with the code. A public repository must not include real patient data, passwords, secrets, or private hospital records.

7 Architecture and Technology

7.1 Current prototype architecture

The baseline is a static, single-browser application. ‘index.html‘ defines the page structure and loads CSS and plain JavaScript. ‘data.js‘ contains the doctor/facility records and demonstration settings; ‘geo.js‘ holds Bangladesh division/district/upazila data. Other scripts implement utilities, browser storage, dropdowns, doctor search/profile, appointment booking, slips, export/backup, administrator actions, status lookup and startup. Browser ‘localStorage‘ stores appointment and patient records; ‘sessionStorage‘ keeps the demo admin session timestamp. No shared backend exists.

Browser UI → JavaScript feature modules → data.js + browser storage

All state remains in this browser profile; it is not synchronized to a clinic.

7.2 Technology selection rationale

Technology	Reason for use / limitation
HTML5	Semantic page layout and native forms with no build step.
CSS3	Responsive cards, typography, layout and visual states.
Vanilla JavaScript	Search and booking logic that can run without a framework; the baseline is modular scripts.
localStorage / sessionStorage	Suitable for a local demo and refresh persistence; unsuitable for shared production records.
SheetJS (optional CDN)	Produces XLSX exports when internet/library loading is available; a CSV fallback exists in the baseline.
jsPDF (optional CDN)	Produces a downloadable appointment-slip PDF when loaded; printing is also available in the baseline.
Graphviz DOT	Editable source for diagrams; PDF, PNG and SVG renderings are provided.
LaTeX / pdfLaTeX	Reproducible academic proposal source and PDF output.

7.3 Production target architecture (future extension)

For a production deployment, the browser should call a server-side API; the API should enforce authentication, authorization, date/slot rules and data validation; a database should store hospitals, doctors, patients, appointments and administrative actions; and a notification gateway may send approved messages. Admin credentials must be hashed and validated server-side, transport should use HTTPS, and sensitive access should be audited. This target architecture is a future recommendation, not a capability delivered by the current static prototype.

8 Data and Domain Model

8.1 Main data entities

- **Doctor:** identifier, hospital, name, department, speciality, days and visit time.
- **Patient:** name, mobile and registration number in the demo flow.
- **Appointment:** reference, doctor/facility association, date, serial, patient, status and creation time.
- **Hospital:** facility name and future verified contact/address fields.
- **Admin user:** a future server-managed account and role; the baseline login is demonstrative only.

8.2 Data quality and privacy

The supplied list is treated as a starter dataset. Before publication as a directory, every doctor/facility relationship, spelling, speciality, clinic address and schedule must be validated by an authorized source, and an owner plus review date should be maintained for each entry. The demo must use fictional patient inputs. In a real system, collection of personal information requires a documented purpose, limited retention, role-based access, transport and storage protections, backup controls, and review against applicable law and institutional policy. Do not use the academic prototype to book medical care.

9 Analysis and Design Diagrams

The diagram set separates the current static prototype from proposed production concepts. DFDs describe the current data movement; the ERD shows a normalized target model for a future backend; the class and object diagrams map directly to the added OOP interface. The DOT source for every diagram is included so it can be edited and re-rendered.

Data Flow Diagram - Context / Level 0

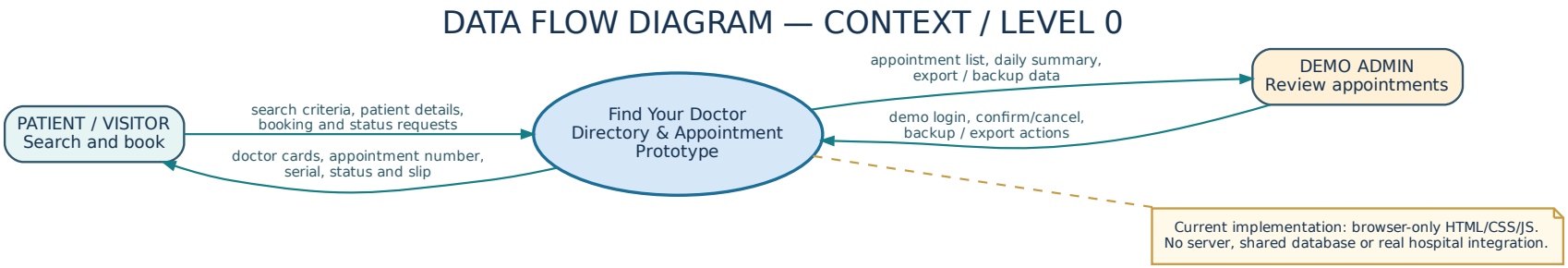


Figure 1: External actors and the overall system boundary. The dotted note records the current browser-only limitation.

Data Flow Diagram - Level 1

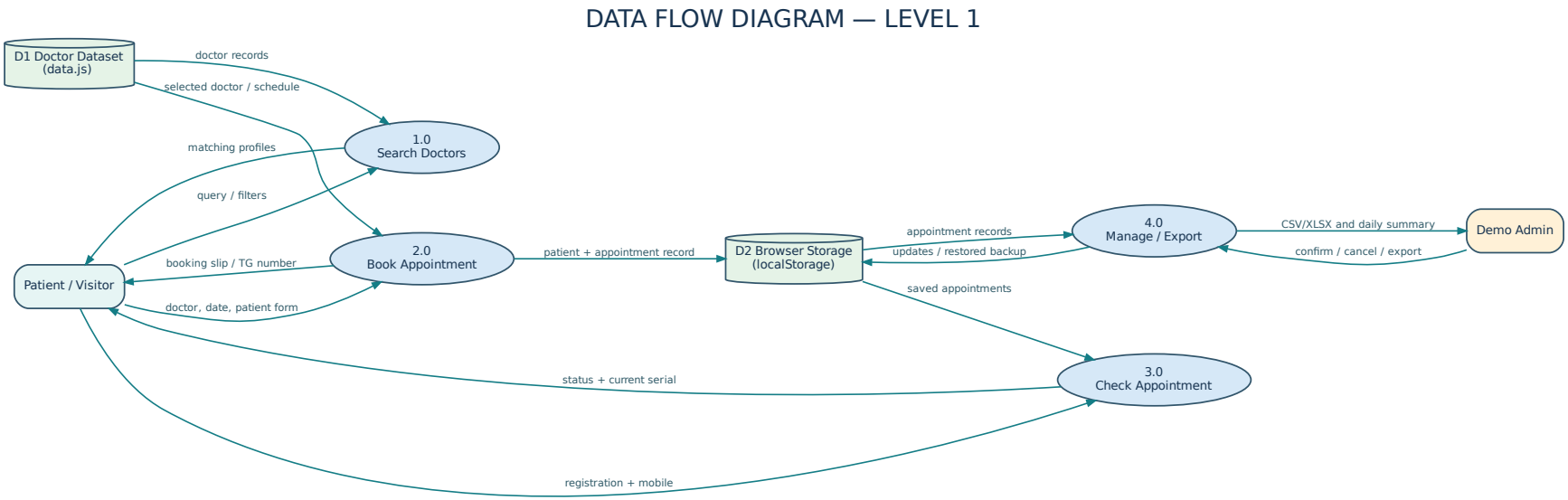


Figure 2: Decomposition of search, booking, status lookup, and demo administration/export. Data stores are the bundled doctor dataset and browser storage.

Use Case Diagram

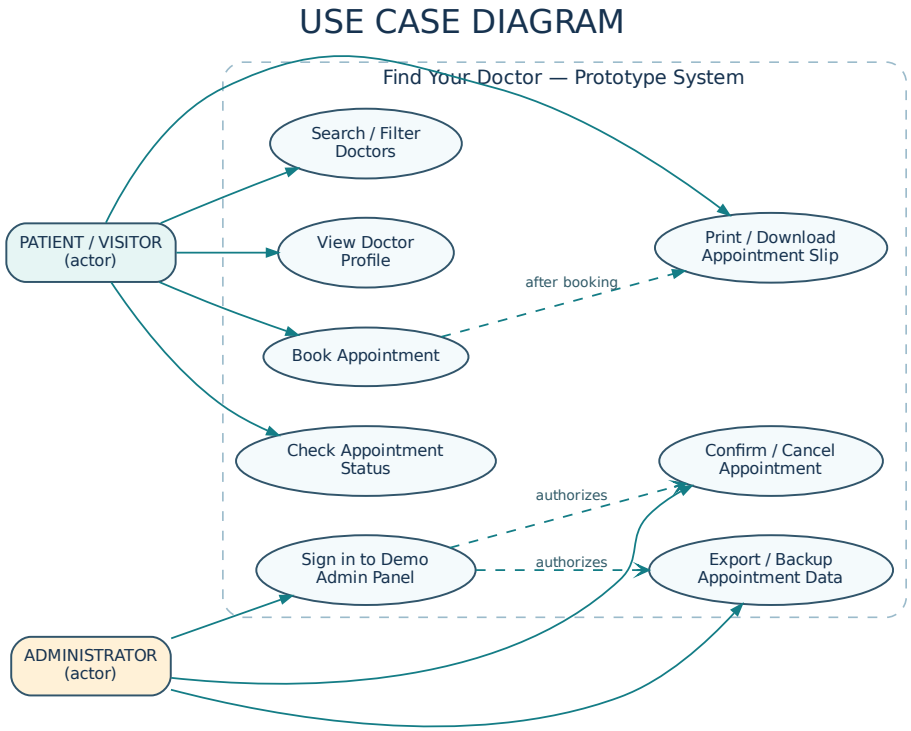


Figure 3: Primary patient/visitor and admin use cases for the submitted prototype. Dashed arrows identify workflow/authorization dependencies, not backend security guarantees.

Activity Diagram - Appointment Booking

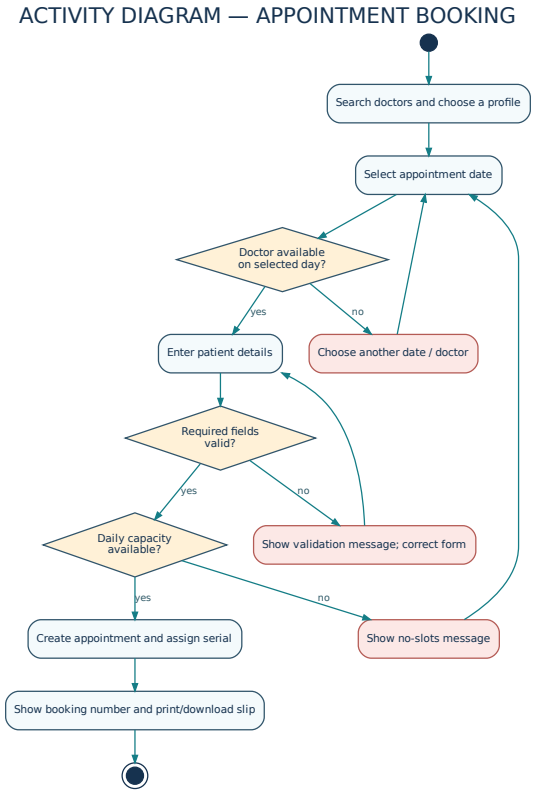


Figure 4: Booking path with availability, form validation, daily capacity, serial generation and failure paths.

Entity-Relationship Diagram

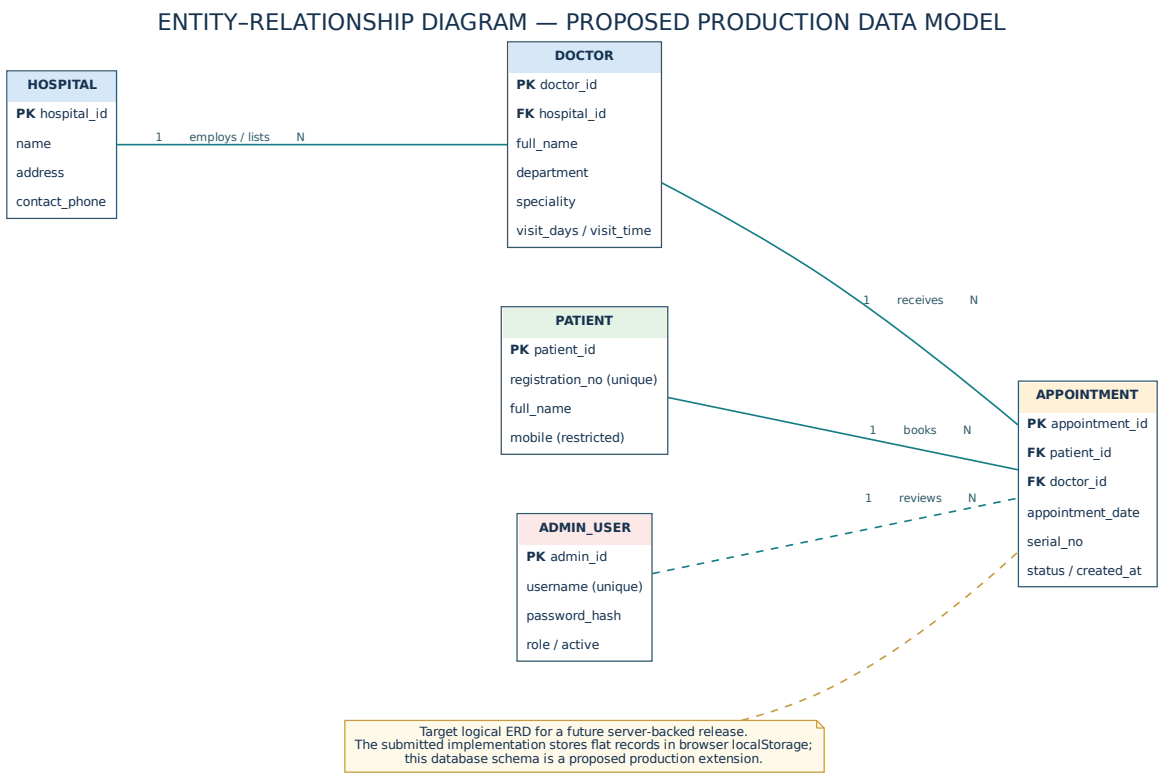


Figure 5: Normalized logical data model recommended for a future server-backed implementation. It is not a claim that these database tables exist in the current prototype.

Class Diagram

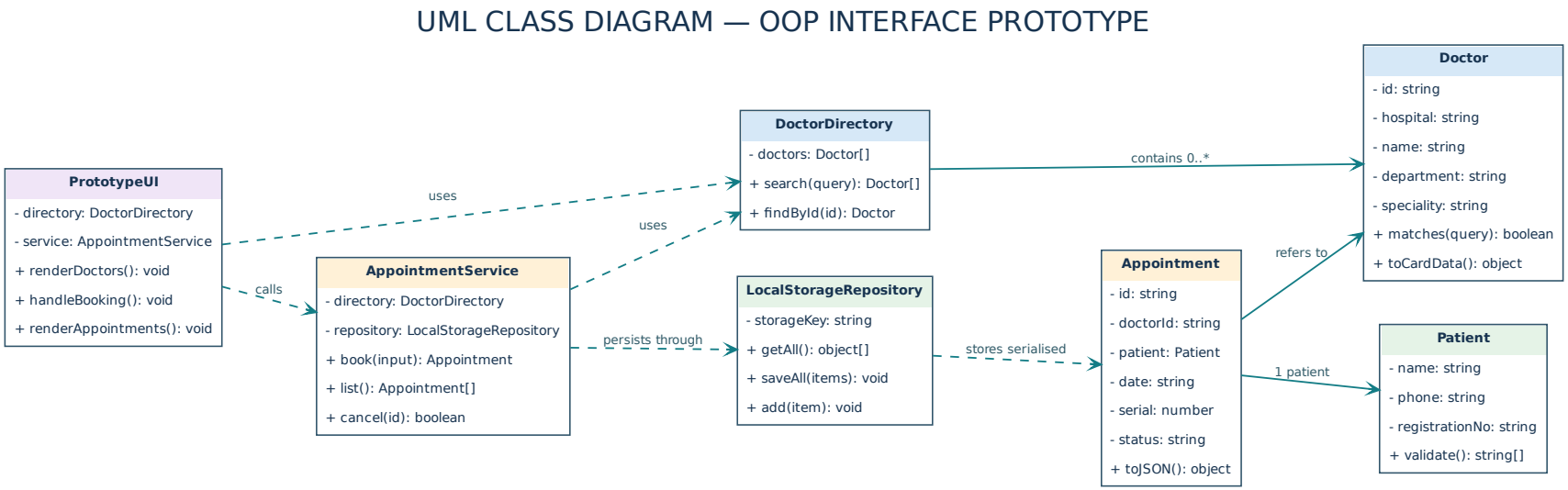


Figure 6: Seven classes implemented in ‘prototype/js/app.js‘: Doctor, Patient, Appointment, DoctorDirectory, LocalStorageRepository, AppointmentService and PrototypeUI.

Object Diagram

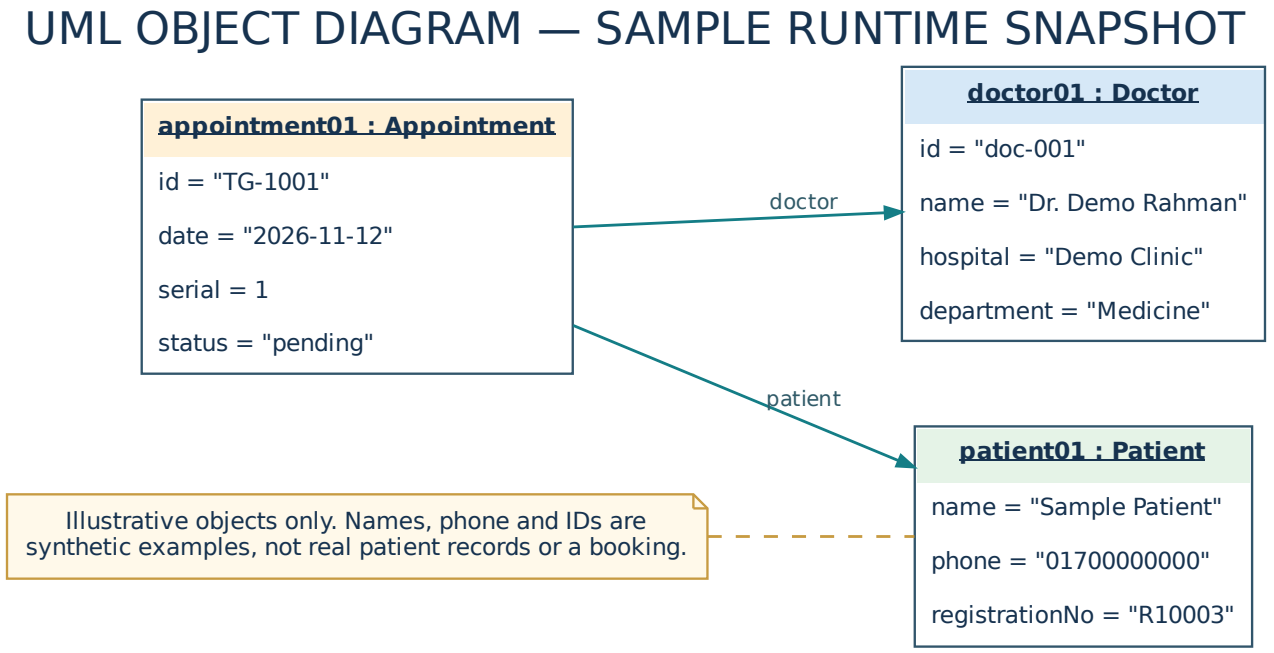


Figure 7: Illustrative runtime objects and links. Sample names, phone and references are synthetic and do not represent a real booking.

10 OOP Interface Prototype

10.1 Implementation classes

The additional interface in ‘prototype/index.html‘ loads the bundled doctor dataset and runs the object-oriented implementation in ‘prototype/js/app.js‘. It contains seven classes (more than the required five), each with a clear responsibility.

Class	Responsibility	Diagram traceability
`Doctor`	Encapsulates doctor/facility fields, search matching and weekday availability.	Doctor entity and methods
`Patient`	Holds submitted patient details and validates name/-mobile input.	Patient association
`Appointment`	Represents a booking reference, doctor, patient, date, serial and status.	Appointment entity
`DoctorDirectory`	Owns the doctor collection and supports search/find-by-ID.	Directory composition
`LocalStorageRepository`	Reads/writes serialized demo appointment records.	Persistence dependency
`AppointmentService`	Enforces date, availability and demo-capacity rules; creates and lists bookings.	Booking/service logic
`PrototypeUI`	Binds events, renders results/messages and connects the interface to services.	UI controller

10.2 How to run the OOP interface

Open the repository folder in VS Code and use Live Server on ‘prototype/index.html‘, or use a local server that serves the repository root. The page loads ‘../js/data.js‘; therefore it should be run from the repository structure rather than copied alone. Search for a doctor, choose a card, enter a synthetic name/mobile/date, then create and review a demo booking. Data persist under a separate ‘localStorage‘ key from the baseline site. Use the Clear button to delete only records created by this OOP demo in the current browser origin.

10.3 Traceability

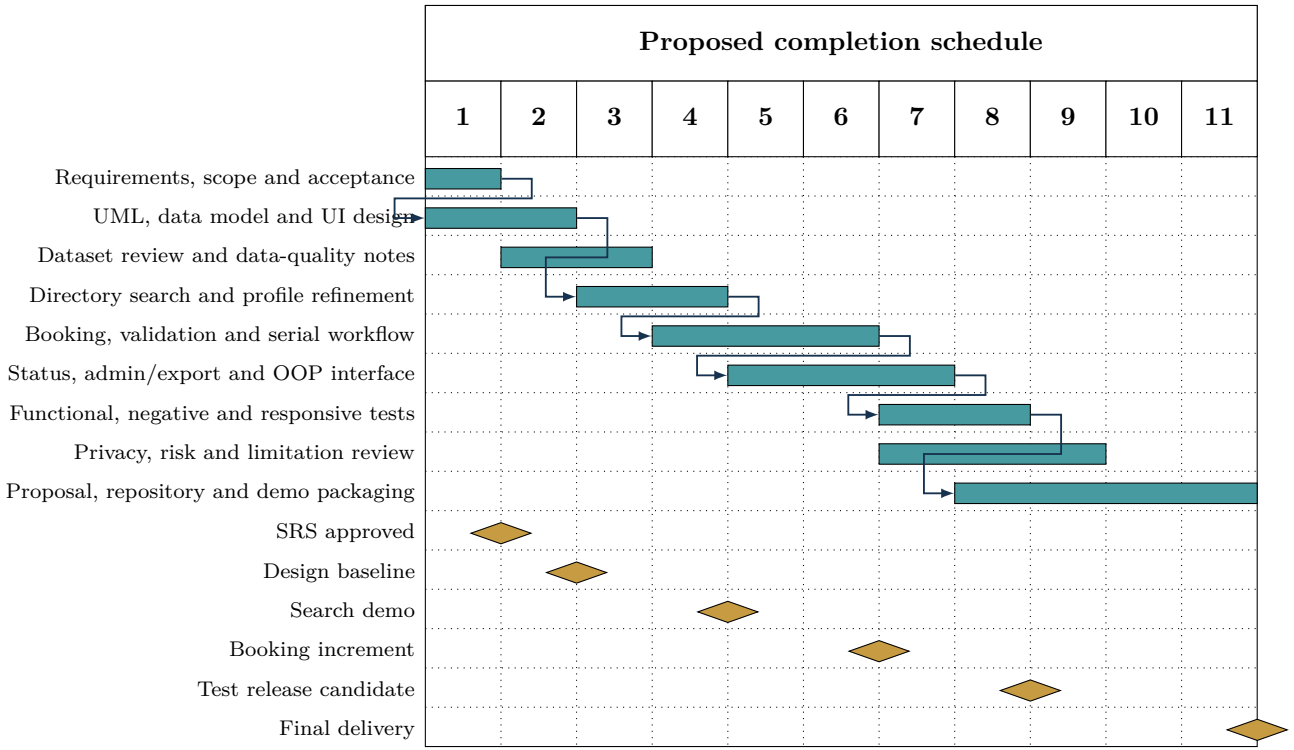
The class diagram lists the same seven class names and their principal methods. The object diagram shows sample instances of Doctor, Patient and Appointment. The interface demonstrates search, validation, booking, local persistence and a list of saved records. The production ERD is intentionally distinct: it describes how the project should be normalized if a server-side database is later implemented.

11 Project Schedule and Gantt Chart

11.1 Planning calendar

The proposed work window is 2 November 2026 to 15 January 2027 (11 planning weeks, with the final week ending on 15 January). Dates are planning assumptions and should be adjusted with the course instructor. Week 1 begins on 2 November 2026. Each week ends with a runnable increment or an inspectable document. The schedule includes the existing baseline prototype; tasks therefore include review, validation, OOP demonstration, refinements and evidence rather than assuming that every feature must be created from zero.

11-week Gantt chart



Week guide: W1 Nov 2-8; W2 Nov 9-15; W3 Nov 16-22; W4 Nov 23-29; W5 Nov 30-Dec 6; W6 Dec 7-13; W7 Dec 14-20; W8 Dec 21-27; W9 Dec 28-Jan 3; W10 Jan 4-10; W11 Jan 11-15.

11.2 Milestones and deliverables

Milestone	Target	Owner	Deliverable / exit condition
M1	Week 1	Student developer	Scope, objectives, stakeholder list, requirements and acceptance criteria reviewed.
M2	Week 2	Student developer	DFD, use case, activity, ERD, class and object diagrams baselined; source folder structure agreed.
M3	Week 4	Student developer	Search/profile interface demonstrable; search, empty state and sample data caveat checked.
M4	Week 6	Student developer	Booking workflow validates input and date, assigns reference/serial and shows a demo pending state.
M5	Week 8	Student developer + peer reviewer	Test plan executed; defects recorded; layout checked on desktop and mobile viewport.
M6	Week 11	Student developer	LaTeX source/PDF, diagrams and sources, README, implementation code, limitations and demo evidence packaged for submission.

11.3 Work breakdown structure

1. **Project initiation:** problem statement, scope, stakeholders, constraints and success criteria.
2. **Analysis/design:** functional/non-functional requirements, data definitions, UML/DFD/ERD diagrams and initial UI flow.
3. **Build:** integrate directory search, booking/status pathways, demo admin/export path and the seven-class OOP interface.
4. **Verification:** test cases, negative input validation, persistence refresh check, responsive layout review and defect fixes.
5. **Release:** repository cleanup, setup guide, source/PDF sync, versioned final package and live demonstration.

11.4 Deliverables register

ID	Deliverable	Contents
D-01	Project proposal	Goals, scope, stakeholders, requirements, process model, constraints and references.
D-02	Design pack	Two DFD views, use-case, activity, ERD, class and object diagrams; editable ‘.dot‘ sources and rendered formats.
D-03	Baseline web prototype	Organized HTML, CSS, data and JavaScript modules; doctor finder and demo appointment workflow.
D-04	OOP interface	Runnable interface implementing seven classes and matching the class diagram.
D-05	Test evidence	Test plan, cases, expected results, observed results and known limitations.
D-06	Reproducible documentation	README, run instructions, code guide, LaTeX file and compiled PDF.
D-07	Final demo package	Clean repository ZIP and source suitable for pushing to a student’s GitHub account.

12 Testing and Quality Assurance

12.1 Test levels

- **Unit/logic checks:** date rejection, doctor availability by encoded weekday, name/mobile validation, serial calculation and storage parsing.
- **Integration checks:** search selection passes the correct doctor ID to the form; successful submission appears in the appointment list; refresh restores the saved list.
- **UI checks:** readable cards, mobile layout, keyboard operation, focus, visible status/error text and no-results state.
- **Regression checks:** run the original baseline site after reorganizing the folders; confirm script paths, image paths, booking/status/admin flows and optional exports.
- **Documentation checks:** verify diagram labels against classes and feature behaviour, compile LaTeX, inspect PDF pages, and make sure repository links are relative and valid.

12.2 Representative test cases

ID	Scenario	Steps / input	Expected result
TC-01	Search match	Search for an existing department or doctor token.	Matching cards are displayed; count updates.
TC-02	Search no result	Search for a deliberately absent token.	Clear no-results message; no broken layout.
TC-03	Missing patient name	Choose doctor/date; submit with name empty.	Browser/form validation blocks submission.
TC-04	Invalid mobile	Enter fewer than 7 digits.	Service returns an input-validation message.
TC-05	Past appointment date	Try a date earlier than today.	Submission is rejected with an actionable message.
TC-06	Unavailable weekday	Choose a doctor whose schedule excludes the selected weekday.	Submission is rejected; no appointment is stored.
TC-07	Successful booking	Submit valid synthetic details for a future listed day.	New reference/serial appears in the saved demo list.
TC-08	Persistence	Refresh the same page/origin after booking.	The demo record remains visible.
TC-09	Separate storage	Compare OOP demo with baseline site local storage keys.	OOP demo records are isolated under their own key.
TC-10	Capacity limit	Seed 15 matching doctor/facility/-date records, then submit one more.	Capacity message; the 16th record is not created.
TC-11	Responsive layout	Inspect at desktop and narrow mobile widths.	Controls remain visible and operable without horizontal overflow.

TC-12	Baseline regression	Open root 'index.html'; try search, booking, status and admin demo.	Local scripts/images load; expected demo flows remain available.

12.3 Defect severity

Critical issues block the demo or risk data corruption; high issues break booking/status flows; medium issues impair a secondary action; low issues are cosmetic or copy. Each defect should be recorded with reproduction steps, expected/actual output, severity and resolution status. No production-readiness claim is made from passing browser-only tests.

13 Risk Management

ID	Risk	Likelihood	Impact	Mitigation / response
R1	Incorrect or outdated doctor/facility information	High	High	Mark data as demo; verify listings with authorized facilities; record source, owner and last-reviewed date before public release.
R2	Local storage is mistaken for a real booking service	Medium	High	Show a persistent prototype disclaimer and pending status; do not advertise the site as an operational service.
R3	Personal data exposure in browser storage or screenshots	Medium	High	Use synthetic details only; never commit real patient data; clear demo storage before public demonstrations.
R4	Admin interface is treated as secure authentication	Medium	High	State limitation; production needs server-side auth, password hashing, authorization, audit trails and session controls.
R5	CDN unavailable so XLSX/PDF function is missing	Medium	Medium	Test the fallback path; document internet dependency; retain browser print/CSV fallback where provided.
R6	Inconsistent serials across browsers/devices	High	High	Explain local-only behaviour; production serial allocation must be transactionally enforced by the server/database.
R7	Schedule slips or scope growth	Medium	Medium	Use weekly increments, define exit criteria and defer external integrations from the initial scope.
R8	Diagrams and implementation drift apart	Medium	Medium	Review traceability at each milestone; keep diagram sources in the same repository and regenerate renderings after edits.

14 Feasibility and Resource Plan

14.1 Technical feasibility

The baseline technologies run in modern browsers without a framework or build system, reducing environment complexity. Graphviz produces editable diagrams and LaTeX produces a reproducible report. The main technical limitation is architectural: local storage is device-local and cannot coordinate appointment capacity across users. A production deployment is feasible only after adding a backend and validating operational workflows.

14.2 Economic feasibility

The academic prototype uses free/open tooling and requires no server hosting to demonstrate locally. Costs for a live service (domain/hosting, managed database, monitoring, messaging, security review and data verification) are not estimated because deployment requirements, expected volume and facility agreements are not yet known. Those costs should be estimated as a separate production feasibility study.

14.3 Operational feasibility

A simple finder and guided booking form can be demonstrated to peers. Real operation would need an assigned data owner, clinic confirmation process, user support, correction requests, incident handling, backup/retention rules and a clear disclaimer for schedule changes. Hospital personnel must validate any operational process before deployment.

14.4 Resources

- Student developer: requirements, coding, diagrams, test execution and documentation.
- Supervisor/peer reviewer: periodic feedback and acceptance review.
- Development tools: VS Code or another editor, browser, Git, Graphviz and pdfLaTeX.
- Test resources: synthetic appointments, multiple browser widths, and a clean browser profile for persistence checks.

15 Limitations and Future Enhancements

15.1 Known limitations of the submitted prototype

- Browser-only persistence; there is no central source of truth, multi-user synchronization, or collision-safe appointment allocation.
- Bundled doctor/hospital listings and schedules are unverified sample data.
- Admin login and appointment lookup are demonstration flows and are not suitable as production security boundaries.
- Optional download/export libraries depend on external CDN availability.
- No SMS, email, payment, hospital management system, or real calendar integration exists.

15.2 Future roadmap

1. Validate and version the doctor/facility dataset with an authorized data steward.
2. Add a backend REST API and relational database with unique constraints and transactional serial allocation.
3. Add secure authentication, role-based authorization, session expiration, rate limiting and audit logs.
4. Add verified notifications and hospital-side appointment confirmation, with explicit consent and reliable contact channels.
5. Add automated tests, CI checks, deployment configuration, monitoring, backup/restore drills and security review.
6. Conduct usability testing with representative users and document outcomes before public release.

16 Ethics, Privacy, and Responsible Use

The project concerns a healthcare-adjacent workflow and can be misunderstood as an official clinical service. The interface and repository therefore identify it as a prototype. Doctor data must not be presented as verified without evidence, and real patient names, phone numbers, registration identifiers, health details or credentials must never be included in source control, sample screenshots or test fixtures. A future deployment must establish an appropriate lawful basis and purpose for data processing, limit access and retention, protect data in transit and at rest, define correction/deletion processes and obtain institutional/legal review. The website must not be used for emergencies or medical decision-making.

17 Repository Structure and Reproducibility

The repository is organized to keep the original static application and the OOP demonstration separate while sharing the bundled doctor dataset.

```
find-your-doctor-capstone/
|-- index.html                  # Baseline application
|-- css/style.css               # Baseline styles
|-- js/                         # Data and feature modules
|-- images/doctor.png
|-- prototype/
|   |-- index.html              # OOP interface
|   |-- css/app.css
|   `-- js/app.js                # Seven implemented classes
|-- docs/diagrams/              # DOT + PDF + PNG + SVG
|-- docs/testing/                # Test plan and manual test record template
|-- proposal.tex                 # Editable report source
|-- proposal.pdf                 # Compiled proposal
|-- README.md
|-- RUN-IN-VSCODE.md
`-- .vscode/
```

17.1 Reproducible run steps

1. Extract the repository ZIP and open its folder in VS Code.
2. Use Live Server on root ‘index.html‘ for the baseline application.
3. Use Live Server on ‘prototype/index.html‘ for the OOP class-based demo.
4. Use synthetic data only. Test on a fresh browser profile when checking first-run behaviour.
5. Compile the report in the repository root with ‘pdflatex proposal.tex‘ (run twice to update the table of contents and references). The precompiled ‘proposal.pdf‘ is included.

18 Conclusion

Find Your Doctor provides a practical capstone scope linking requirements engineering, interface design, core application logic, testing, and repository delivery. The current static application is a useful demonstration baseline, while the additional OOP interface fulfills the class-based design requirement with seven explicit classes. The diagrams cover data flow, user goals, control flow, logical data design, class structure and example objects. The 11-week incremental plan establishes reviewable milestones and deliverables. The most important boundary is that a browser-only academic demonstration is not a real booking platform: verified data, shared server-side processing, secure authentication, operational agreement and privacy review would be prerequisites for production use.

19 References

1. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
2. Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1.* https://www.omg.org/spec/UML/2.5.1
3. ISO/IEC/IEEE. (2018). *ISO/IEC/IEEE 29148:2018 Systems and software engineering - Life cycle processes - Requirements engineering.*
4. Project code and accompanying guide supplied with this capstone work. The guide describes a browser-only HTML/CSS/JavaScript implementation with localStorage persistence and optional client-side export libraries.

A Submission Checklist

☐ Proposal PDF compiles, opens and has no clipped diagrams or headings.

☐ Editable LaTeX source is included in the GitHub repository.

☐ DFD, use-case, activity, ERD, class and object diagrams are available as PDF/PNG/SVG with editable DOT files.

☐ OOP prototype demonstrates at least five classes from the class diagram.

☐ README explains both baseline and OOP demo launch paths.

☐ The limitations and synthetic-data disclaimer are visible and clear.

☐ Manual test results are filled in before final submission.

☐ Repository is pushed to GitHub and its public URL is pasted into the course submission page.